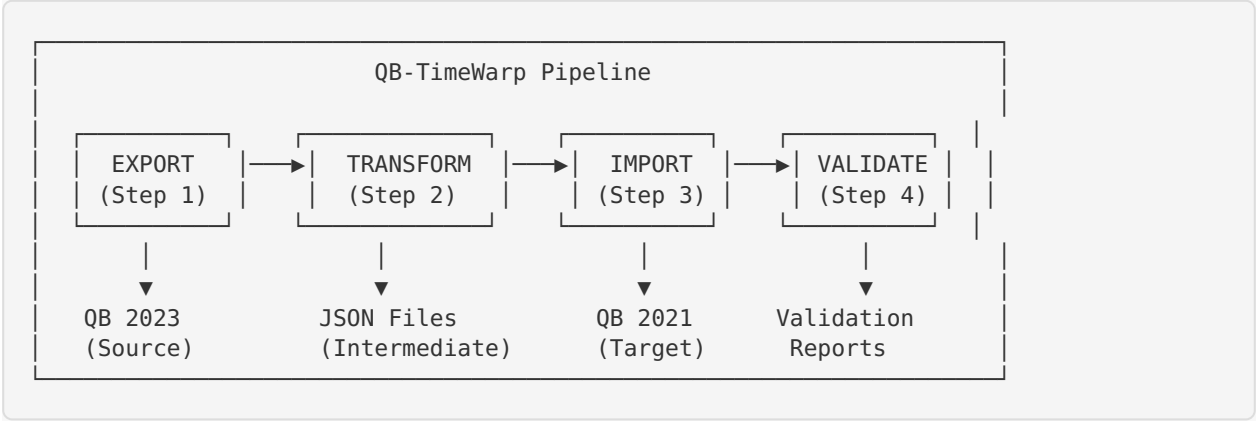
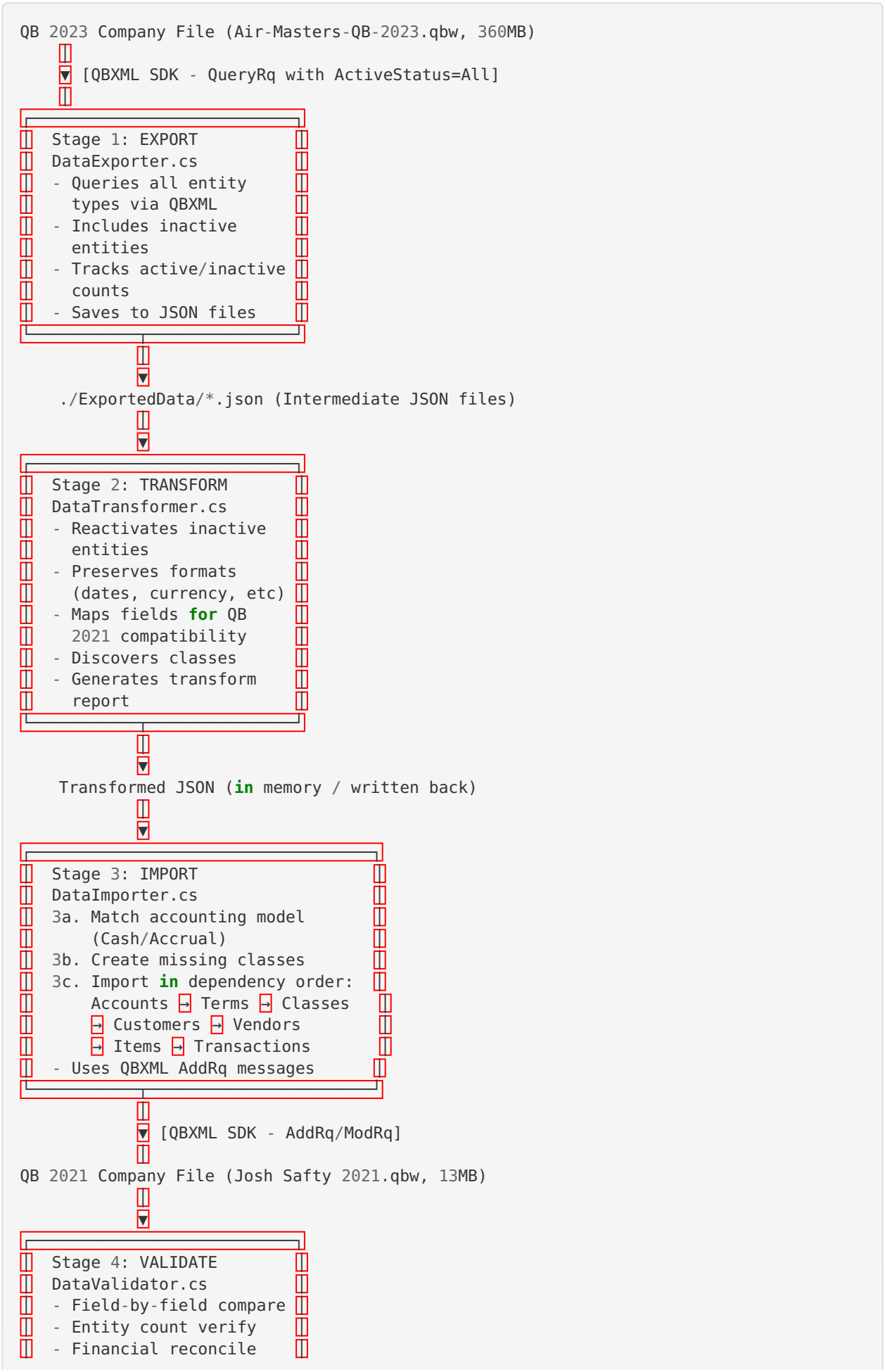
QB-TimeWarp — System Architecture

High-Level Architecture



Data Flow Detail


QB 2023 Company File (Air-Masters-QB-2023.qbw, 360MB)
[QBXML SDK - QueryRq with ActiveStatus=All]
Stage 1: EXPORT
DataExporter.cs
- Queries all entity types via QBXML
- Includes inactive entities
- Tracks active/inactive counts
- Saves to JSON files
./ExportedData/*.json (Intermediate JSON files)
Stage 2: TRANSFORM
DataTransformer.cs
- Reactivates inactive entities
- Preserves formats (dates, currency, etc)
- Maps fields for QB 2021 compatibility
- Discovers classes
- Generates transform report
Transformed JSON (in memory / written back)
Stage 3: IMPORT
DataImporter.cs
3a. Match accounting model (Cash/Accrual)
3b. Create missing classes
3c. Import in dependency order:
Accounts → Terms → Classes
→ Customers → Vendors
→ Items → Transactions
- Uses QBXML AddRq messages
[QBXML SDK - AddRq/ModRq]
QB 2021 Company File (Josh Safty 2021.qbw, 13MB)
Stage 4: VALIDATE
DataValidator.cs
- Field-by-field compare
- Entity count verify
- Financial reconcile

Service Layer Breakdown

`QBConnectionManager.cs` — QuickBooks SDK Connection

Responsibility: Manages the COM Interop connection to QuickBooks via QBXML SDK.

- Opens/closes sessions with QuickBooks
- Sends QBXML request messages and receives responses
- Handles connection errors, timeouts, and retries
- Manages the SDK version negotiation (16.0 for QB 2023, 15.0 for QB 2021)

Key Methods:
- `Connect(config)` — Establishes COM connection to QB
- `ProcessRequest(qbxmlRequest)` — Sends QBXML and returns response XML
- `Disconnect()` — Cleanly closes the session

Important: Must target x86 due to 32-bit COM SDK. QuickBooks must be running with the company file open.

`DataExporter.cs` — Export from QB 2023

Responsibility: Queries all entity types from QB 2023 and saves them as JSON.

- Builds QBXML query requests for each entity type
- Adds `<ActiveStatus>All</ActiveStatus>` for entity types that support it (Customers, Vendors, Items, Employees, Accounts, etc.)
- Converts QBXML responses to JObjects
- Tracks active vs. inactive entity counts
- Saves export manifest with statistics

Key Methods:
- `ExportAll()` — Orchestrates full export
- `ExportEntityType(type)` — Exports a single entity type
- `ConvertXmlToEntity(xml)` — Parses QBXML response to JSON
- `SaveExportManifest()` — Writes export statistics

Entity Types Supporting ActiveStatus Filter:
Accounts, Customers, Vendors, Employees, Items, PaymentMethods, Terms, Classes, SalesTaxCodes, ShipMethods, CustomerTypes, VendorTypes, JobTypes, PriceLevels

---

`DataTransformer.cs` — Transform for QB 2021 Compatibility

Responsibility: Applies all transformation rules to make QB 2023 data compatible with QB 2021.

Transformation Pipeline:
1. **Reactivation**: Sets `IsActive = true` on all entities (if enabled)
2. **Field Mapping**: Applies field name/type mappings from `FieldMappings.json`
3. **Format Preservation**: Detects and preserves formats for dates, currency, phones, postal codes, memos
4. **Class Discovery**: Scans transactions for class references, builds list of required classes
5. **Value Processing**: Truncates fields to QB 2021 max lengths, handles type conversions

Key Methods:
- `TransformAll(exportedData)` — Orchestrates full transformation
- `ProcessValue(value, mapping)` — Format-aware value transformation
- `DetectFormatType(fieldName, value)` — Auto-detects date/currency/phone/postal/memo
- `ApplyFormatPreservation(value, formatType, mapping)` — Dispatches to format-specific handlers
- `BuildTransformationReport()` — Generates summary statistics

Configuration: Reads from both `appsettings.json` (TransformationRules) and `FieldMappings.json` (formatRules).

`DataImporter.cs` — Import into QB 2021

Responsibility: Imports transformed data into QB 2021 via QBXML SDK.

Import Sequence (dependency order):
1. Apply accounting preferences (Cash/Accrual matching)
2. Create missing classes
3. Import reference entities (Accounts, Terms, PaymentMethods, etc.)
4. Import party entities (Customers, Vendors, Employees)
5. Import item entities
6. Import transaction entities (Invoices, Bills, Payments, JournalEntries, etc.)

Key Methods:
- `ImportAll(transformedData)` — Orchestrates full import
- `EnsureClassesExist(requiredClasses)` — Creates missing classes in QB 2021
- `QueryExistingClasses()` — Fetches current classes from QB 2021
- `CreateClassInQB2021(className)` — Creates a single class (handles parent:child hierarchy)
- `QueryCompanyPreferences()` — Reads QB 2021 company preferences
- `ApplyAccountingPreferences(sourcePrefs)` — Updates QB 2021 accounting method

`DataValidator.cs` — Post-Import Validation

Responsibility: Comprehensive validation of migrated data.

Validation Checks:
1. **Entity Count Verification** — Source count vs. target count for each entity type
2. **Field-by-Field Comparison** — Compare individual field values between source and target
3. **Financial Reconciliation** — Verify financial totals match (invoices, payments, balances)

4. **Journal Validation** — Verify every journal entry balances (debits = credits ± tolerance)
5. **Format Preservation Validation** — Verify dates, currency, phones, postal codes match expected formats

Key Methods:
- `ValidateAll(sourceData, importedData)` — Orchestrates all validation
- `ValidateFormatPreservation(source, imported)` — Format-specific validation
- `ValidateDateFieldFormats(fields, ...)` — Checks QBXML date format compliance
- `ValidateCurrencyFieldFormats(fields, ...)` — Checks decimal precision
- `ValidatePhoneFieldFormats(fields, ...)` — Checks length limits (max 21 chars)
- `ValidatePostalCodeFieldFormats(fields, ...)` — Checks length limits (max 13 chars)
- `BuildSummary()` — Generates human-readable validation summary

`SchemaExtractor.cs` — QB 2021 Schema Discovery

Responsibility: Extracts field definitions from QB 2021 to understand target constraints.

- Queries sample records from QB 2021 for each entity type
- Infers field names, data types, and constraints
- Falls back to `KnownSchemas` for entity types with no sample data
- Saves schemas as JSON for reference

Key Methods:
- `ExtractAllSchemas()` — Extracts schemas for all entity types
- `ExtractEntitySchema(type)` — Extracts schema for one entity type
- `DiscoverFieldsFromXml(element)` — Recursively discovers fields from QBXML
- `InferDataType(value)` — Infers QBXML data type from sample values

`TransformFunctions.cs` — Format Utility Functions

Responsibility: Static utility functions for all format-specific operations.

Function Groups:
- **Date**: `DetectDateFormat()`, `PreserveDate()`, `ValidateDateForQB2021()`, `IsDateField()`
- **Currency**: `PreserveCurrencyFormat()`, `IsCurrencyField()`
- **Phone**: `PreservePhoneFormat()`
- **Postal Code**: `PreservePostalCodeFormat()`
- **Memo**: `PreserveMemoFormat()`
- **Generic**: `FormatAwareTruncate()`

`ProgressReporter.cs` — UI Helpers

Responsibility: Console output formatting and progress tracking.

- `ProgressReporter` class — Real-time progress bars with ETA
- `ConsoleBanner` class — Color-coded headers, steps, success/warning/error messages

---

Configuration Files

`appsettings.json`

Section	Purpose
`QuickBooks.QB2023`	Source QB connection (path, SDK version, timeouts)
`QuickBooks.QB2021`	Target QB connection
`Paths`	Output directories for exports, schemas, logs, validation
`Export`	Export settings (batch size, inactive records, entity types)
`Import`	Import settings (batch size, error handling, import order)
`Validation`	Validation toggles and tolerance
`TransformationRules`	Reactivation, class tracking, accounting model toggles
`Logging`	Serilog configuration

`Configuration/FieldMappings.json`

Section	Purpose
`fieldMappings`	Per-entity-type field name/type/length mappings
`formatRules`	Format preservation toggles and patterns
`_documentation`	Inline documentation of all settings

Model Layer

Model File	Key Classes	Purpose
`ConfigurationModels.cs`	`AppConfiguration`, `TransformationRulesConfig`, `ValidationConfig`, `ExportConfig`, `ImportConfig`	Configuration binding
`MigrationModels.cs`	`ExportedEntitySet`, `TransformationReport`, `MigrationReport`, `CompanyPreferences`, `ClassTrackingSummary`, `FormatPreservationStats`	Runtime data models
`ValidationModels.cs`	`ValidationReport`, `FormatValidationReport`, `FormatValidationIssue`	Validation results
`FieldMappingModels.cs`	`FieldMappingsConfig`, `FieldMapping`, `FormatRulesConfig`	Field mapping configuration
`QBEntityType.cs`	Entity type enums/constants	Entity type definitions

How Transformation Rules Work

1. **Configuration Loading**: `Program.cs` reads `appsettings.json` → binds to `TransformationRulesConfig`
2. **Rule Injection**: Config is passed to `DataTransformer` constructor
3. **Per-Entity Processing**: For each entity in each entity type:
   - If `ReactivateInactiveEntities`: Set `IsActive = true`, increment counter
   - For each field: Check `FieldMappings.json` for mapping rules
   - Auto-detect format type (date/currency/phone/postal/memo)
   - Apply format-specific preservation via `TransformFunctions`
   - Track all transformations in `TransformationReport`
4. **Class Discovery**: Scan transaction fields for `ClassName`/`ClassRef`
5. **Report Generation**: Build summary of all transformations applied

How Validation Works

1. **Source Snapshot**: Export data serves as source-of-truth
2. **Target Query**: After import, re-query QB 2021 to get actual imported data
3. **Comparison**: For each entity type and entity:
   - Count verification (source count = target count)
   - Field-by-field value comparison (with tolerance for amounts)

   - Journal balance check (sum of debits = sum of credits per journal)
   - Format compliance (dates in yyyy-MM-dd, currency with 2 decimals, etc.)
4. **Report**: Generate JSON report with pass/fail per check, detailed issues list